

Paradigmas (O que é, Imperativos e Declarativos)

Um **paradigma de programação** define a forma como os programadores estruturam e organizam a lógica do código. Em outras palavras, é um modelo ou estilo (ou ainda, uma filosofia =D) de programação que guia como resolver problemas e organizar programas.

Por exemplo, em *POO* (programação orientada a objetos) "os programadores podem abstrair um programa como uma coleção de objetos que interagem entre si", enquanto na *programação funcional* o foco são as funções puras. Outras classificações incluem o paradigma **procedural, imperativo, funcional, reativo**, entre outros. Muitas linguagens modernas suportam múltiplos paradigmas (por exemplo, C# suporta POO, imperativo, genérico etc.).

Definição: Segundo a literatura, "*paradigma de programação é um meio de se classificar as linguagens baseado em suas funcionalidades*". Em termos práticos, o paradigma determina a visão de como estruturar e executar o programa. Duas abordagens clássicas são:

Paradigmas Imperativo vs Declarativo:

Paradigma Imperativo: Foca no "como" fazer as tarefas, especificando sequências de comandos que alteram o estado do programa. Por exemplo, um programa imperativo diz ao computador "faça isso, depois aquilo". Trata a computação como uma série de instruções que modificam variáveis.

Paradigma Declarativo: Foca no "o que" deve ser feito, sem detalhar passo a passo o fluxo de controle. Em vez de descrever comandos, o programador define regras, consultas ou expressões e o sistema determina a melhor forma de realizá-las. Em termos formais, "programação declarativa se concentra em o que o programa deve realizar sem especificar todos os detalhes de como".

Exemplo comparativo: Suponha listar números pares até 10.

- Imperativo (exemplo em C#):



```
1  for(int i=1; i<=10; i++) {  
2      if(i % 2 == 0) Console.WriteLine(i);  
3  }
```

- Declarativo (exemplo em LINQ/C#):



```
1  var pares = Enumerable.Range(1, 10).Where(n => n % 2 == 0);  
2  foreach(int n in pares) Console.WriteLine(n);
```

O primeiro diz *como* iterar e verificar; o segundo descreve *o que* filtrar (todos os pares).

Tabela comparativa:

Aspecto	Imperativo	Declarativo
Estilo	Passo a passo (comandos sequenciais)	Especificação de resultados desejados
Foco	Como alterar estado (variáveis, memória)	O quê resolver (regras, consultas)
Controle	Estruturas (loops, condicionais, atribuições)	Expressões, funções puras, consultas
Exemplos	C, C++, Java (POO), Assembly	SQL, HTML, Prolog, linguagens funcionais (Haskell), XSLT
Vantagem	Controle fino do processo, fácil de aprender	Mais conciso, evita bugs de fluxo
Desvantagem	Código mais verboso, propenso a erros de estado	Às vezes menos intuitivo de depurar

Imperativo inclui tanto a *programação procedural* (sequencial, com funções) quanto estilos estruturados. **Declarativo** abrange paradigmas como *lógico*, *funcional* e *reativo*, onde o programador descreve propriedades e deixa que o sistema as realize.

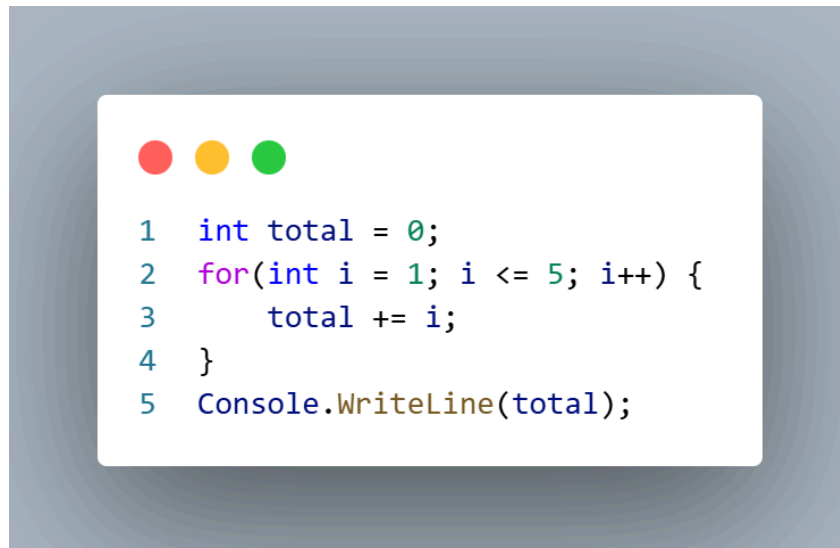
As duas principais famílias (Imperativo e Declarativo) e seus paradigmas

Paradigmas Imperativos:

Definição: Segundo a literatura, “programação imperativa é um paradigma de programação que descreve a computação como ações, enunciados ou comandos que mudam o estado (variáveis) do programa”. Programadores imperativos especificam algoritmos detalhados, operando em variáveis mutáveis.

- **Programação Procedural:** Subparadigma imperativo clássico, baseado em procedimentos (funções) e execução sequencial. Usa estruturas de controle (loops, condicionais) e possivelmente variáveis globais. Por exemplo, C puro, Fortran e Pascal são procedurais.

- **Programação Estruturada:** Ênfase em controle de fluxo bem definido (sequência, seleção, iteração) sem goto. A Programação Estruturada (PE) usa subrotinas (funções/métodos) e estruturas condicionais/loops bem formadas. Foi o paradigma dominante antes da POO.
 - Exemplo (C# estruturado):



```
1  int total = 0;
2  for(int i = 1; i <= 5; i++) {
3      total += i;
4  }
5  Console.WriteLine(total);
```

- **Características:** Paradigma imperativo (procedural/estruturado) enfatiza **como** executar cada passo. É semelhante ao modo como falamos naturalmente (ordens/imperativos). Muitas linguagens populares (C, C#, Java, Python, etc.) suportam esse estilo.
- **Programação Orientada a Objetos (POO):** Paradigma que modela o software como objetos que representam entidades do mundo real, cada um com dados e comportamentos. É “um paradigma baseado no conceito de ‘objetos’, que podem conter dados na forma de atributos e códigos na forma de métodos”. Objetos interagem entre si, trocando mensagens ou chamando métodos. A POO facilita a organização de sistemas complexos em unidades autônomas.

Conceitos-chave da POO:

- **Classe:** um molde (protótipo) que define o formato de dados (*atributos*) e comportamentos (*métodos*) de um tipo de objeto. Por exemplo, `class Pessoa { public string Nome; public int Idade; public void Falar() { ... } }`. A classe especifica o estado e o que o objeto pode fazer.
- **Objeto (ou Instância):** Um exemplar concreto de uma classe, alocado na memória em tempo de execução. Cada objeto tem valores próprios para os atributos da classe. Em C#, cria-se com o operador new. Exemplo:



```
1 Pessoa p = new Pessoa();  
2 p.Nome = "Ana";  
3 p.Idade = 25;  
4 p.Falar();
```

Nesse código, `p` é um **objeto** da classe `Pessoa`. “Um objeto é uma instância de uma classe, ou seja, a materialização de uma classe na memória. Cada objeto pode ter valores diferentes para seus atributos, mas compartilha os mesmos métodos definidos pela classe.”.

- **Atributos (Campos/Propriedades):** Variáveis definidas em uma classe, que armazenam o estado do objeto. No exemplo acima, `Nome` e `Idade` são atributos. “Atributos são variáveis declaradas dentro de uma classe que armazenam o estado de um objeto.”.
- **Métodos:** Funções definidas em uma classe que descrevem comportamentos do objeto. No exemplo, `Falar()` é um método da classe `Pessoa`. “Métodos representam comportamentos ou ações que um objeto pode executar.”.
- **Instanciação:** É a operação de criar um objeto a partir de uma classe, usando `new`. Cada chamada a `new` invoca o **construtor** (método especial) da classe. “Instanciar um objeto é criar uma instância de uma classe na memória usando o operador `new` em C#.”.

- **Vantagens dos Paradigmas Imperativos:** Controle detalhado do fluxo, fácil de entender algoritmicamente para iniciantes. Pode ser mais eficiente em tarefas simples, pois o programador define exatamente o processo.
- **Desvantagens dos Paradigmas Imperativos:** Em projetos grandes, o código pode virar “espaguete” se não for bem organizado. É suscetível a bugs relacionados ao estado compartilhado (por exemplo, alterações inesperadas em variáveis).

Paradigmas Declarativos:

Definição: A programação declarativa “concentra-se no que o programa deve realizar sem especificar todos os detalhes de como”. O programador define regras, relações ou expressões, e a execução é determinada por um mecanismo de inferência, compilador ou ambiente de runtime.

- **Programação Lógica:** Utiliza **lógica matemática** (normalmente lógica de predicados) para expressar fatos e regras, e um motor de inferência resolve consultas. Por exemplo, em Prolog,

define-se um banco de fatos ("Socrates é humano") e regras ("todo humano morre"), e pergunta-se "Socrates morre?" O sistema deduz a resposta. Em resumo, "programação lógica é um paradigma que faz uso da lógica matemática".

- *Exemplo (Prolog):*



```
1  homem(socrates).  
2  mortal(X) :- homem(X).  
3  ?- mortal(socrates).  
4  % Resposta: true.
```

- **Programação Funcional:** Trata a computação como avaliação de *funções matemáticas* puras, evitando mutabilidade de dados. Não há (ou há mínima) alteração de estado; o foco é no resultado de chamadas de função. Exemplos: Haskell, Lisp, F#. "Programação funcional [...] enfatiza a aplicação de funções, em contraste com a programação imperativa". Funções são cidadãos de primeira classe (podem ser parâmetros e retornos).

- *Exemplo em C# (LINQ/funcional):*



```
1  var nums = new int[] {1,2,3,4,5};  
2  var pares = nums.Where(x => x % 2 == 0);  
3  Console.WriteLine(string.Join(", ", pares));
```

- **Programação Reativa:** Paradigma **orientado a fluxos de dados e eventos**, enfatizando programação assíncrona e não bloqueante. Os programas reativos reagem a mudanças em *streams* de dados (eventos, mensagens) em vez de executar passos sequenciais. Todas as operações são propagadas por esses fluxos. Isso permite alta escalabilidade em aplicações

assíncronas. O manifesto reativo formaliza esse conceito (Responsividade, Resiliência, Elasticidade e Orientação a Mensagens).

- *Resumo*: "Paradigma reativo é orientado a fluxo de dados e propagação de estados, usando streams assíncronos".

- **Outros Paradigmas Declarativos:** (Menções breves) Por exemplo, *Programação em Regra* (usada em alguns sistemas especialistas) e *programação baseada em restrições* (CSP, Constraint Logic), entre outros. O foco comum é descrever propriedades do problema, não os passos para solucioná-lo.

Resumo dos paradigmas:

Paradigma	Natureza	Exemplo / Linguagem	Características principais
Imperativo	Foco no <i>como</i>	C, C++, Java (POO), Python	Sequências de comandos, mutabilidade
Estruturado	Subtipo imperativo	C (sem goto)	Seqüência, seleção, iteração sem goto
Procedural	Subtipo imperativo	C, Pascal	Uso extensivo de funções/procedimentos
Declarativo	Foco no <i>o quê</i>	SQL, HTML	Descreve resultado, não processo
Lógico	Declarativo (inferência)	Prolog	Fatos e regras (lógica de predicados)
Funcional	Declarativo (funções)	Haskell, Lisp, F#	Funções puras, expressões, sem estado mutável
Reativo	Declarativo/Imperativo mix	RxJS, Akka (frameworks)	Streams de dados, eventos assíncronos

Comparativo: Programação Procedural vs. POO

Embora seja imperativo, a programação **procedural** e a **orientada a objetos** têm abordagens diferentes:

- **Procedural (Imperativo):** Concentra-se em funções/rotinas que operam em dados. O código está organizado em procedimentos sequenciais; dados e funções são separados.
- **Orientado a Objetos:** Organiza o código em objetos que combinam *dados* e *comportamentos*. Em POO, os dados e as funções que os manipulam (métodos) residem dentro das *classes*. Isso encapsula o estado e facilita a reutilização em sistemas grandes.

Prós/Contras:

- *Procedural:* Simplicidade inicial, fácil de aprender e adequado para tarefas específicas (scripts, algoritmos científicos). Porém, tende a levar a **baixo reaproveitamento** e dificuldade de manutenção em sistemas grandes.
- *OOP:* Altamente modular (cada classe é autocontida), facilita manutenibilidade e reutilização. Adequada para projetos complexos (ex.: sistemas corporativos, jogos). Requer maior entendimento conceitual e adiciona overhead estrutural.

Quando usar cada um: Para *problemas pequenos ou cálculos diretos*, o procedural é suficiente e até mais performático. Para *sistemas robustos que evoluem*, a POO oferece melhor organização.

Comparação de Códigos Procedural vs POO:

Veja abaixo um exemplo lado a lado em C# calculando a área de um retângulo:



```
1 // Procedural (função solta em C#)
2 double CalcularArea(double largura, double altura) {
3     return largura * altura;
4 }
5 double areaProc = CalcularArea(5.0, 3.0);
6 Console.WriteLine($"Área (procedural): {areaProc}");
7
```



```
1 // Orientação a Objetos (definindo uma classe)
2 class Retangulo {
3     public double Largura;
4     public double Altura;
5     public double Area() {
6         return Largura * Altura;
7     }
8 }
9 // Uso da classe Retangulo
10 Retangulo r = new Retangulo();
11 r.Largura = 5.0;
12 r.Altura = 3.0;
13 double areaOOP = r.Area();
14 Console.WriteLine($"Área (OOP): {areaOOP}");
15
```

No código procedural, escrevemos uma função `CalcularArea` que atua nos dados passados. Na versão orientada a objetos, criamos uma classe `Retangulo` que contém *atributos* (`Largura`, `Altura`) e um *método* `Area()`. Cada abordagem produz o mesmo resultado, mas a OOP agrupa dados e funções em unidades coerentes (objetos).

Vantagens, Desvantagens e Anti-Padrões da POO

Vantagens e Desvantagens da POO:

Vantagens (POO):

- **Modularidade e Reutilização:** Cada classe é autocontida; muitas instâncias podem ser criadas. Isso facilita reaproveitar código em outros projetos.
- **Manutenção facilitada:** Alterar ou estender uma classe não costuma afetar outras partes do sistema, especialmente se existir *encapsulamento* adequado. Projetos grandes ficam mais organizados e “gerenciáveis”.
- **Modelagem natural:** Reflete entidades do mundo real (ex.: clientes, produtos, pedidos), facilitando a compreensão do domínio.
- **Pilares da OOP:** Encapsulamento, Herança e Polimorfismo provêm recursos poderosos para estruturar e estender sistemas.

Desvantagens (POO):

- **Complexidade inicial:** Existe uma curva de aprendizado maior: é preciso entender vários conceitos (classe, objeto, herança, etc.).
- **Sobrecarga de desempenho:** Criar e gerenciar objetos exige mais recursos de memória/CPU (embora em muitas aplicações isso seja negligível).
- **Nem sempre necessário:** Para tarefas muito simples ou algoritmos puros (e.g. cálculo matemático isolado), usar POO pode ser exagero. Em alguns cenários, a POO adiciona verbosidade sem muito ganho prático.

Anti-padrões e Cuidados Comuns:

Mesmo em POO, há *má práticas* que prejudicam o código. Alguns exemplos clássicos:

- **God Object (Objeto Deus):** Uma única classe que faz “tudo”, acumulando muito responsabilidade (muitos métodos e atributos). Torna o sistema rígido e difícil de manter.
- **Anemic Domain Model (Modelo Anêmico):** Classes que só contêm atributos públicos e *getters/setters*, sem lógica interna, transferindo toda a lógica para fora das classes. Isso quebra o encapsulamento e dispersa as regras de negócio.
- **Excesso de Herança:** Criar hierarquias profundas desnecessárias, quando composição seria melhor. Uso abusivo de herança múltipla pode causar confusão na resolução de métodos.
- **Falta de Encapsulamento:** Expor atributos como public indiscriminadamente, sem proteger o estado do objeto. Preferir sempre métodos (ou propriedades) para controlar acesso.
- **Código Duplicado:** Ignorar classes e reutilização, copiando e colando funcionalidades em várias partes.
- **Baixa Coesão:** Ter classes que não têm foco claro (realizam múltiplas tarefas não relacionadas).

Evitar esses anti-padrões é crucial para um código orientado a objetos de qualidade. Usar conceitos como *SRP* (Single Responsibility Principle) e *DRY* (Don't Repeat Yourself) ajuda a manter o design limpo.

Exercícios práticos =D

1. Defina com suas palavras o que são (a) **classe**, (b) **objeto** e (c) **método** na POO. Dê um exemplo simples em C# de cada um.
2. Dado o código procedural abaixo que calcula a média de duas notas, reescreva-o **usando orientação a objetos** (crie classe Calculadora, por exemplo).



```
1  double CalcularMedia(double n1, double n2) {  
2      return (n1 + n2) / 2.0;  
3  }  
4  double media = CalcularMedia(7.5, 8.0);  
5  Console.WriteLine($"Média: {media}");  
6
```

GABARITO/POSSÍVEIS RESPOSTAS:

1. *Resposta:* (a) Uma classe é um modelo ou protótipo que define atributos e métodos (por exemplo, class Carro { ... }). (b) Um objeto é uma instância de uma classe (por exemplo, Carro c = new Carro();). (c) Um método é uma função definida na classe (por exemplo, c.Ligar() é um método do objeto c).
2. *Resposta:* Uma possível solução em POO:



```
1  class Calculadora {  
2      public double CalcularMedia(double n1, double n2) {  
3          return (n1 + n2) / 2.0;  
4      }  
5  }  
6  // Uso:  
7  Calculadora calc = new Calculadora();  
8  double media = calc.CalcularMedia(7.5, 8.0);  
9  Console.WriteLine($"Média (POO): {media}");  
10
```

Note que agora CalcularMedia é um método da classe Calculadora.